NEO4J • STREAMLIT • GRAPH DB

DogRecommet

ระบบแนะนำสายพันธุ์สุนัขด้วย Graph Database ที่อธิบายเหตุผลเบื้องหลังคำแนะนำได้แบบเรียลไทม์

ผู้จัดทำ

นางสาวภิญนิตา ซิ่วมณี 664245042

เว็บแอปพลิเคชัน

dogrecommet.streamlit.app



01 • ARCHITECTURE

โครงสร้างและการทำงานของระบบ

1. ผู้ใช้เลือกความชอบ

เลือกสายพันธุ์สุนัขที่ชื่นชอบผ่านหน้าเว็บแอปพลิเคชัน

2. Streamlit & Python

ส่งคำสั่ง Cypher Query ประมวลผลตรรกะคำนวณคะแนน

3. Neo4j Aura Cloud

ค้นหาความสัมพันธ์เชิงกราฟ อัปเดตข้อมูลแบบทันทีทันใด

Security & Real-time: ปลอดภัยด้วยการเก็บ credentials ใน st.secrets และเขียนอัปเดตตรงลง Neo4j Cloud ทุกครั้งที่มีการแก้ไข

02 • GRAPH MODEL

โครงสร้างกราฟและโหนดข้อมูล

DogLover (Node)

ผู้ใช้งาน

เก็บข้อมูลชื่อผู้ใช้ (name)

DogBreed (Node)

สายพันธุ์สุนัข

เก็บ name, size, energy, condo_ok, emoji, note

BreedGroup (Node)

กลุ่มสายพันธุ์

เก็บชื่อกลุ่มสายพันธุ์ (name)

ความสัมพันธ์และข้อกำหนด

RELATIONSHIPS

- [:LIKES] : DogLover → DogBreed
- [:IN_GROUP] : DogBreed → BreedGroup

CONSTRAINTS

กำหนด Uniqueness Constraint เพื่อป้องกันข้อมูลซ้ำซ้อนเมื่อทำการบันทึกข้อมูลหลายครั้ง

03 • SAMPLE DATA

สรุปข้อมูลตัวอย่างในระบบ

5	8	4	11
ผู้ใช้ (DogLover)	สายพันธุ์ (DogBreed)	กลุ่มสายพันธุ์	ความชอบ (LIKES)

ตัวอย่างการจัดกลุ่ม (Breed Groups)

Toy: Pomeranian, Chihuahua

Herding: Corgi, Border Collie

Retriever: Golden Retriever, Labrador

Spitz: Shiba Inu, Siberian Husky

ตัวอย่างความชอบผู้ใช้ (User Likes)

Mild: Pomeranian, Corgi

Ten: Pomeranian, Corgi, Shiba Inu

Pim: Corgi, Shiba Inu

Arm & Bow: Golden, Labrador, Shiba Inu

04 • RECOMMENDATION PATHS

2 เส้นทางหลักในการหาคำแนะนำ

Path 1: คนที่มีความชอบคล้ายกัน (Collaborative Filtering)

น้ำหนัก × 2

ค้นหาผู้ใช้อื่นที่ชอบสุนัขพันธุ์เดียวกับเรา แล้วแนะนำพันธุ์อื่นที่ผู้ใช้นั้นชอบ

[Mild] → LIKES → **[Corgi]** ← LIKES ← **[Pim]** → LIKES → **[Shiba Inu]**

Path 2: อยู่ในกลุ่มสายพันธุ์เดียวกัน (Content-Based)

น้ำหนัก × 1

แนะนำสุนัขที่อยู่ในกลุ่มสายพันธุ์เดียวกับสายพันธุ์ที่เรากดชอบ

[Mild] → LIKES → **[Pomeranian]** → IN_GROUP → **[Toy]** ← IN_GROUP ← **[Chihuahua]**

05 • SCORING & RULES

หลักการคิดคะแนนและเงื่อนไขพิเศษ

คะแนนรวม = (จำนวนเส้นทาง Path 1 × 2) + (จำนวนเส้นทาง Path 2 × 1)

ตัวอย่างคำนวณสำหรับผู้ใช้ "Mild" (ชอบ Pomeranian, Corgi)

สายพันธุ์	Path 1 (×2)	Path 2 (×1)	คะแนนรวม
Shiba Inu	3 เส้นทาง	0 เส้นทาง	**6 คะแนน**
Border Collie	0 เส้นทาง	1 เส้นทาง	**1 คะแนน**
Chihuahua	0 เส้นทาง	1 เส้นทาง	**1 คะแนน**

ผู้ใช้ใหม่ (Cold Start)

หากยังไม่มีประวัติความชอบ ระบบจะแนะนำสายพันธุ์ยอดนิยมแทน

ไม่แนะนำซ้ำ (Deduplication)

ใช้คำสั่ง NOT EXISTS กรองสายพันธุ์ที่ผู้ใช้กดชอบแล้วออก

โหมดคอนโด (Condo Filter)

ตัวเลือกกรองเฉพาะสายพันธุ์ที่ระบุว่า condo_ok = true

06 • EXPLAINABLE RECOMMENDATION

ความโปร่งใส: อธิบายเหตุผลได้ทุกคำแนะนำ

#1 • 6 คะแนน

🐶 Shiba Inu

กลุ่ม Spitz • Medium • พลังงาน Medium • 🏢 คอนโดได้

👥 **Ten, Pim** ชอบ Corgi, Pomeranian เหมือนคุณ และชอบพันธุ์นี้ด้วย **(3 เส้นทาง × 2)**

#2 • 1 คะแนน

🐕 Chihuahua

กลุ่ม Toy • Small • พลังงาน Low • 🏢 คอนโดได้

🧬 อยู่กลุ่ม **Toy** เดียวกับ Pomeranian ที่คุณชอบ **(1 เส้นทาง × 1)**

* เหตุผลสร้างขึ้นจาก ใน CypherQuery จริงตรงจากโครงสร้างกราฟ ไม่ใช่ข้อความที่สุ่มมา
collect(DISTINCT o.name)

07 • CYPHER QUERY

คำสั่ง Query หลักเบื้องหลังการค้นหา

// Path 1: คนที่ชอบคล้ายกัน

```
MATCH (me:DogLover {name:$n}) -[:LIKES]->(via:DogBreed) <-[:LIKES]-(o:DogLover) -[:LIKES]->(rec:DogBreed) WHERE o <> me AND NOT EXISTS {(me)-[:LIKES]->(rec)} RETURN rec.name AS breed, count(*) AS paths, collect(DISTINCT o.name) AS people
```

MATCH (me:DogLover {name:$n}) -[:LIKES]->(via:DogBreed) <-[:LIKES]-(o:DogLover) -[:LIKES]->(rec:DogBreed) WHERE o <> me AND NOT EXISTS {(me)-[:LIKES]->(rec)} RETURN rec.name AS breed, count(*) AS paths, collect(DISTINCT o.name) AS people

// Path 2: กลุ่มสายพันธุ์เดียวกัน

```
MATCH (me:DogLover {name:$n}) -[:LIKES]->(via:DogBreed) -[:IN_GROUP]-> (g:BreedGroup) <-[:IN_GROUP]-(rec:DogBreed) WHERE NOT EXISTS {(me)-[:LIKES]-> (rec)} RETURN rec.name AS breed, count(*) AS paths, collect(DISTINCT g.name) AS groups
```

MATCH (me:DogLover {name:$n}) -[:LIKES]->(via:DogBreed) -[:IN_GROUP]-> (g:BreedGroup) <-[:IN_GROUP]-(rec:DogBreed) WHERE NOT EXISTS {(me)-[:LIKES]-> (rec)} RETURN rec.name AS breed, count(*) AS paths, collect(DISTINCT g.name) AS groups

08 • SYSTEM FEATURES

เมนูฟังก์ชันการใช้งานระบบ

Dashboard

สรุปภาพรวมจำนวนข้อมูล และจัดอันดับสายพันธุ์ยอดนิยมแบบ Real-time

แนะนำสุนัข

เลือกสายพันธุ์ที่ชอบแล้วแสดงการ์ดคำแนะนำพร้อมอธิบายเหตุผล

ผู้ใช้ (Users)

บริหารจัดการผู้ใช้งาน ดูรายการความชอบ และลบผู้ใช้ในระบบ

สายพันธุ์

เพิ่ม แก้ไข และลบข้อมูลสายพันธุ์สุนัขพร้อมระบุกลุ่มสายพันธุ์

Graph Explorer

แสดงแผนภาพเครือข่ายกราฟแบบโต้ตอบ พร้อมไฮไลต์คำแนะนำ

Setup Menu

สร้างข้อมูลตัวอย่างตั้งต้น (Initial Data) เข้าสู่ระบบได้ทันทีด้วยปุ่มเดียว

เริ่มต้นใช้งาน DogRecommet

ระบบแนะนำที่โปร่งใส ตรวจสอบเหตุผลได้ และขับเคลื่อนด้วยพลังของ Graph Database

WEB APPLICATION
dogrecommet.streamlit.app

GITHUB REPOSITORY
github.com/Phinnita/DogRecommet

Powered by Python • Streamlit • Neo4j Aura • Cypher Query